University of Asia Pacific

Department of Computer Science and Engineering

# Peer-to-Peer Network Communication and File Sharing Programming Assignment

Objective:
Build a lightweight peer-to-peer network in Python where multiple students can connect directly to one another and exchange text messages and files without using a central server.

Course Code: CSE 433
Course: Blockchain & Distributed Security Lab Programming
Language: Python
Communication: TCP Sockets
Assignment Type: Implementation Project
Deadline: TBA

Department of Computer Science and Engineering
University of Asia Pacific
CSE — P2P Network Assignment UAP

## 1. Introduction

Modern distributed applications often require multiple computers to communicate directly with each other. One important model for such communication is the Peer-to-Peer (P2P) model. In

a traditional client-server architecture, clients communicate through a central server:

Client A −→ Central Server ←− Client B

The server acts as the central point through which communication takes place.

In a P2P architecture, each participating computer can act as both a client and a server. Peers can communicate directly:

Peer A ←→ Peer B

For this assignment, you will implement a small P2P network using Python TCP sockets.

Each student will run one peer on their own computer. Peers will connect to one another using an IP address and a port number. After establishing a connection, they will be able to exchange text messages and files.

> The main purpose of this assignment is not to build a complicated application. The goal is to understand how computers communicate directly over a network using sockets, TCP, threads, and application- level protocols.

## 2. Assignment Objectives

After completing this assignment, students should be able to:

1. Explain the basic idea of a peer-to-peer network.
2. Explain the difference between client-server and P2P communication.
3. Understand IP addresses and port numbers.
4. Create a TCP server using Python sockets.
5. Create a TCP client using Python sockets.
6. Establish direct communication between two peers.
7. Handle multiple peer connections using threads.
8. Design a simple application-level communication protocol.
9. Send and receive text messages.
10. Transfer binary files over a TCP connection.
11. Handle images, audio, video, and other ordinary files.
12. Handle basic network errors and disconnected peers.
13. Demonstrate a working multi-peer network.



## 3. Problem Statement

You are required to develop a lightweight P2P communication application in Python. Each

running in stance of the application represents one peer.

Every peer must be capable of performing two roles:

1. Server role: accept incoming connections from other peers.
2. Client role: initiate connections to other peers.

Therefore, a peer should not depend on a central server. A simplified network may look like:

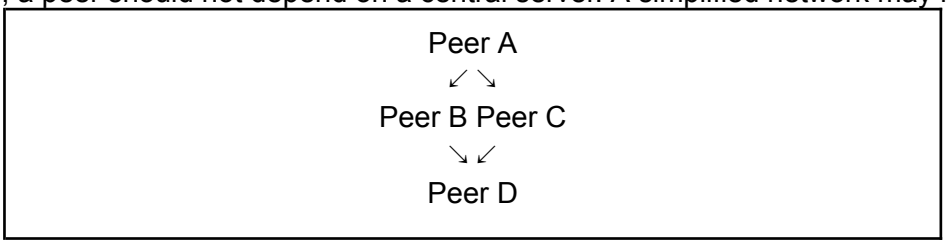



Figure 1: Example P2P network

Peers should be able to establish direct TCP connections and exchange information without sending their messages through a central server.

## 4. Why Are You Doing This Assignment?

This assignment connects several important concepts that are often studied separately. For example:

IP Address
↓
Port Number
↓
Socket
↓
TCP Connection
↓
Application Protocol
↓
Text / File Communication

Instead of simply studying these concepts theoretically, you will implement them. The assignment also provides an introduction to the networking concepts behind many distributed systems. The same fundamental idea of nodes communicating over a network appears in systems such as dis tributed databases, blockchain networks, distributed file systems, and other decentralized applications.

> You are NOT required to implement a blockchain or cryptocurrency for this assignment. The focus is specifically on understanding P2P networking and direct communication between peers.

## 5. Required Background

Before beginning the implementation, students should understand the following

5.1 IP Address

An IP address identifies a computer or network interface on a network. For example:

```
192.168.1.10
```

When two computers are connected to the same local network, they may communicate using their local IP addresses.

5.2 Port Number

A port identifies a particular network service/application on a computer. For example:

```
192.168.1.10:5000
```

Here:

- 192.168.1.10 is the IP address.
- 5000 is the port number.

5.3 Socket

A socket provides an interface through which applications can communicate over a network. For TCP communication, the basic process is:

```
Server:
    socket()
    bind()
    listen()
    accept()

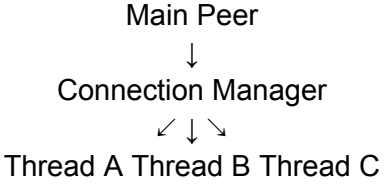
Client:
    socket()
    connect()
```

5.4 TCP

TCP provides reliable, ordered communication between two endpoints. This is useful for our assignment because files must arrive without missing or reordered bytes.

5.5 Threads

A peer must be able to communicate with more than one peer. Therefore, a separate thread can be used to handle each connection. For example:



## 6. System Requirements

### 6.1 Software Requirements

- Python 3.9 or later.
- A code editor such as VS Code, PyCharm, or another IDE.
- Windows, Linux, or macOS.
- Python's built-in socket library.
- Python's built-in threading library.
- Tkinter may be used for the graphical interface.

No third-party Python packages are required for the basic implementation.

### 6.2 Hardware / Network Requirements

For testing on one computer:

- One computer.
- Multiple terminal/application instances.

For the final demonstration:

- Preferably 3–4 computers.
- All computers connected to the same LAN/Wi-Fi network.

## 7. Required Functionality

Your application must provide the following functionality.

### 7.1 Peer Startup

A user should be able to start a peer by specifying:

- Peer name.
- Listening port.

Example:

```
Name: Alice
Port: 5000
```

The peer should then start listening for incoming connections.

### 7.2 Peer Connection

A user should be able to connect to another peer using:

- IP address.
- Port number.

Example:

```
IP: 192.168.1.20
Port: 5001
```



### 7.3 Connected Peer List

The application should display the peers currently connected to the local peer. For example:

```
Connected Peers

Alice [a83f21c4]
Bob [92bd71e3]
Charlie [c8e42a91]
```

### 7.4 Text Communication

A peer should be able to send a text message to another connected peer. Example:

```
Alice -> Bob:
Hello Bob!
```

Bob should receive the message without requiring a central server.

### 7.5 File Transfer

A peer should be able to select a file and send it to another connected peer. The implementation must support ordinary binary files including:

- Text files.
- Images.
- Audio files.
- Video files.
- PDF files.
- ZIP files.
- Other ordinary files.

The application should not implement a separate transfer protocol for each file type. Instead, the file should be treated as binary data.

### 7.6 Received Files

Received files should be stored locally. For example:

```
downloads/
    photo.jpg
    song.mp3
    lecture.mp4
```

## 8. System Architecture

The application consists of three main components.

1. User Interface.
2. P2P Networking Layer.



3. Communication Protocol.

A simplified architecture is:

User Interface
↓
P2P Node
↓
TCP Socket
↓
Other Peer

The P2P node must contain both:

- a server socket for accepting connections;
- client functionality for connecting to other peers.

Therefore:

Every peer = TCP Server + TCP Client

## 9. Recommended Project Structure

Students are recommended to organize their project as follows:

```
P2P_Network/
 |
 |-- main.py
 |-- p2p_node.py
 |-- protocol.py
 |-- requirements.txt
 |
 |-- downloads/
 |
 '-- README.md
```

The responsibilities of the files are:

| File | Responsibility |
|---|---|
| main.py | User interface and interaction with the user. |
| p2p node.py | Peer networking, connections, text communication, and file transfer. |
| protocol.py | Encoding and decoding application-level messages. downloads/ Stores files received from other peers. |
| README.md | Project setup, usage instructions, and explanation. |

Students may organize their code differently if the required functionality is maintained.

## 10. Implementation Steps

The project should be implemented incrementally. Do not attempt to implement the complete system at once.



### 10.1 Step 1: Create a TCP Server

First create a basic TCP server. The server should:

1. Create a socket.
2. Bind it to an IP address and port.
3. Start listening.
4. Accept a connection.

Basic structure:

```
server_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

server_socket.bind(("0.0.0.0", 5000))
server_socket.listen()

connection, address = server_socket.accept()
```

### 10.2 Step 2: Create a TCP Client

Next create a client that connects to the server.

```
client_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

client_socket.connect(("127.0.0.1", 5000))
```

At this stage, students should verify that two programs can establish a TCP connection.

### 10.3 Step 3: Exchange a HELLO Message

After connecting, peers should introduce themselves. The message may contain:

```
{
    "type": "hello",
    "peer_id": "a83f21c4",
    "peer_name": "Alice",
    "port": 5000
}
```

The purpose of this message is to allow each peer to know who it is communicating with. The peer that receives the HELLO message should respond with a HELLO acknowledgement.

```
{
    "type": "hello_ack",
    "peer_id": "92bd71e3",
    "peer_name": "Bob",
    "port": 5001
}
```



The HELLO exchange is called a handshake. It establishes the identity information needed by the application before normal communication begins.

### 10.4 Step 4: Handle Connections Using Threads

A peer must remain available to accept additional connections while communicating with existing peers. Therefore, each connection should be handled independently. Conceptually:

```
while server_is_running:

    connection = accept()

    create_new_thread(connection)
```

This allows multiple peers to communicate concurrently.

### 10.5 Step 5: Design the Message Protocol

The peers need to distinguish between different types of data. At minimum,

implement: • hello

- hello ack
- text
- file

A text message may look like:

```
{
    "type": "text",
    "sender_id": "a83f21c4",
    "sender_name": "Alice",
    "message": "Hello Bob!"
}
```

A file message should contain metadata:

```
{
    "type": "file",
    "sender_id": "a83f21c4",
    "sender_name": "Alice",
    "filename": "photo.jpg",
    "filesize": 2456789
}
```

### 10.6 Step 6: Implement Message Framing

TCP is a byte-stream protocol. It does not automatically preserve application-level message boundaries. Therefore, your program must define where one message ends and another begins. One simple solution is to send the size of a JSON message first. The format is:

[4-byte length][JSON payload]

For example:



```
+----------------+----------------------+
| 4-byte length | SON message|
+----------------+----------------------+
```

The receiver first reads the 4-byte length and then reads exactly that number of bytes.

### 10.7 Step 7: Implement Text Messaging

Once the protocol works, implement text communication. The sender should:

1. Select a connected peer.
2. Enter a message.
3. Create a text message object.
4. Send the message.

The receiver should:

1. Receive the message.
2. Identify the sender.
3. Display the message.

10.8 Step 8: Implement File Transfer

File transfer is the main technical part of the assignment.
A file consists of bytes rather than JSON text.
Therefore, the transfer should happen in two stages.
First send metadata:

```
{
    "type": "file",
    "filename": "video.mp4",
    "filesize": 5242880
}
```

Then send the raw file bytes.
Conceptually:

File Metadata
↓
Raw File Bytes
↓
Receiver

The receiver reads exactly the specified number of bytes and writes them into a file. 9



10.9 Step 9: Use Chunks for Large Files

Do not attempt to read a very large file into memory all at once.
Instead, read and send it in chunks.
For example:

```
while True:
    chunk = file.read(64 * 1024)

    if not chunk:
        break

    socket.sendall(chunk)
```

The receiver performs the corresponding operation until all expected bytes have been received.

10.10 Step 10: Implement Multiple Peers

Finally, allow the local peer to maintain multiple connections.

For example:

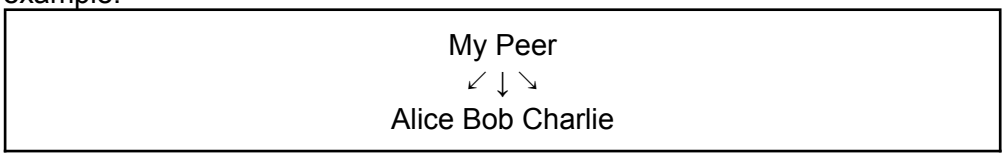



Figure 2: One peer communicating with multiple peers

The user should be able to select a peer and send a message or file specifically to that peer.

## 11. File Transfer Protocol

Students should understand the following sequence.
Suppose Alice wants to send photo.jpg to Bob.

Sender

Alice determines:

```
filename = photo.jpg
filesize = 2456789 bytes
```

Alice first sends:

```
{
    "type": "file",
    "filename": "photo.jpg",
    "filesize": 2456789
}
```

Alice then sends:

```
[raw bytes of photo.jpg]
```



Receiver

Bob receives the metadata and learns:

```
filename = photo.jpg
filesize = 2456789
```

Bob then reads exactly:

2456789

bytes from the TCP connection.
Those bytes are written to:

```
downloads/photo.jpg
```

The receiver must use the file size from the metadata to determine when the file transfer is com plete. It should not simply assume that one call to recv() represents one complete file.

## 12. User Interface Requirements

A graphical interface is recommended but should remain simple.

The interface should provide at least:

1. Peer name input.
2. Listening port input.
3. Start Peer button.
4. Stop button.
5. Remote IP input.
6. Remote port input.
7. Connect button.
8. Connected peer list.
9. Text message input.
10. Send button.
11. Choose File and Send button.
12. Communication/event log.

A possible layout is:

```
-----------------------------------------------------
| My Peer |
| Name: [Alice] Port: [5000] [Start Peer] |
-----------------------------------------------------
| Connect to Another Peer |
| IP: [192.168.1.20] Port: [5001] [Connect] |
-----------------------------------------------------
```



```
| Connected Peers | Messages / Events |
| | |
| Alice | Bob -> You: Hello |
| Bob | You -> Bob: Hi! |
| Charlie | File received: photo.jpg |
-----------------------------------------------------
| Send Text: [_____] [Send] |
-----------------------------------------------------
| [Choose File & Send] |
```

---

Students may design their own interface.
The interface design itself is not the primary focus of the assignment.

## 13. Testing Requirements

Testing must be performed progressively.

### 13.1 Test 1: Same Computer

Run two copies of the application.

Peer A:

```
Name: Alice
Port: 5000
```

Peer B:

```
Name: Bob
Port: 5001
```

Bob connects to:

```
IP: 127.0.0.1
Port: 5000
```

Test:

- Connection.
- Text message.
- Image transfer.
- Audio transfer.
- Video transfer.

### 13.2 Test 2: Two Computers

Connect two computers to the same Wi-Fi/LAN.

Example:

```
Computer A:
IP = 192.168.1.10
Port = 5000

Computer B:
IP = 192.168.1.11
Port = 5001
```

Computer B should connect to:

```
192.168.1.10:5000
```

### 13.3 Test 3: Multiple Peers

At least three peers should be used.

Example:

```
Alice -> 5000
Bob -> 5001
Charlie -> 5002
```

Test:

- Alice communicating with Bob.
- Bob communicating with Charlie.
- Charlie communicating with Alice.
- File transfer between different pairs.

## 14. Basic Error Handling

The application should handle common errors gracefully.

Examples include:

- Invalid IP address.
- Invalid port.
- Peer is not running.
- Connection refused.
- Peer disconnects unexpectedly.
- File does not exist.
- Invalid file size.
- Attempting to send without selecting a peer.

The application should not crash simply because another peer disconnects.
Instead, it should display an appropriate message.

For example:

```
[ERROR] Connection failed: Connection refused
```



## 15. Scope of the Assignment

This is intentionally a lightweight educational P2P project. The following are not required: • Blockchain.

- Cryptocurrency.
- Smart contracts.
- Proof of Work.
- Proof of Stake.
- Distributed consensus.
- DHT.
- BitTorrent protocol.
- NAT traversal.
- WebRTC.
- End-to-end encryption.
- Authentication.
- Database systems.
- Cloud deployment.
- Complex routing algorithms.
- File deduplication.
- File chunk recovery.

## 16. Required Deliverables

Each student must submit the whole project as a zip file.

### 16.1 1. Source Code

The complete Python source code. The project should run using:

```
python main.py
```

### 16.2 2. README

The README must contain:

- Project description.
- Requirements.
- Installation/setup instructions.
- How to run the application.

- How to connect two peers.
- How to transfer files.
- Example screenshots.

### 16.3 3. Demonstration

Students must demonstrate:

1. Starting a peer.
2. Connecting to another peer.
3. Sending a text message.
4. Sending an image.
5. Sending an audio or video file.
6. Connecting multiple peers.
7. Communication between multiple peers.

## 17. Submission Format

Submit a ZIP file using the following naming convention:

ID Section P2P Assignment.zip

Example:

```
22101001_A2_P2P_Assignment.zip
```

The ZIP file should contain:

```
22101001_3A_P2P_Assignment/
 |
 |-- main.py
 |-- p2p_node.py
 |-- protocol.py
 |-- requirements.txt
 |-- README.md
 |
 '-- downloads/
```

## 18. Marking Rubric

Total marks: 20

| Component | Marks | Evaluation |
|---|---|---|
| Peer startup and configuration | 2 | Correct peer name, port, and server startup. |
| TCP connection and Peer identification | 5 | Correct direct connection between two peers, HELLO handshake, and connected peer information. |

Text messaging 3 Correct text communication between peers.
File transfer 5 Correct transfer of text, image, audio, and video/binary files.
Multiple peers 5 Ability to maintain and communicate with multiple peers.
Viva 10 Full understanding of the project. Total 30

## 19. Demonstration Checklist

During the demonstration, students should be prepared to show the

following. 1. Start Peer A.

2. Start Peer B.

3. Connect Peer B to Peer A.

4. Show the connected peer list.

5. Send a text message from A to B.

6. Send a text message from B to A.

7. Send an image from A to B.

8. Send an audio file from B to A.

9. Send a video file from A to B.

10. Start Peer C.

11. Connect Peer C to the network.

12. Demonstrate communication involving Peer C.

Example Figures:

## 20. Questions Students Should Be Able to Answer

During the demonstration or viva, students may be asked:

1. What is the difference between a client and a server?

2. In this project, why can one peer act as both a client and a server?

3. What is the purpose of an IP address?

4. What is the purpose of a port number?

5. Why do different peers need different ports when testing multiple peers on the same

computer? 6. What happens when connect() is called?

7. What happens when accept() is called?

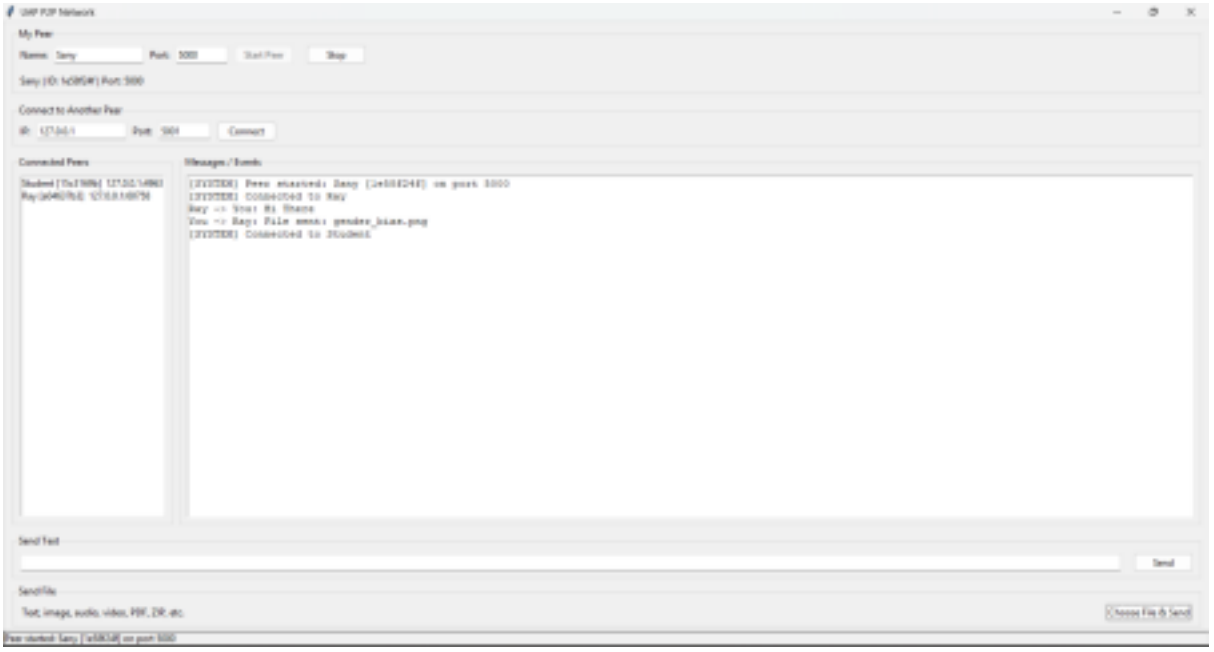


Figure 3: Peer 1

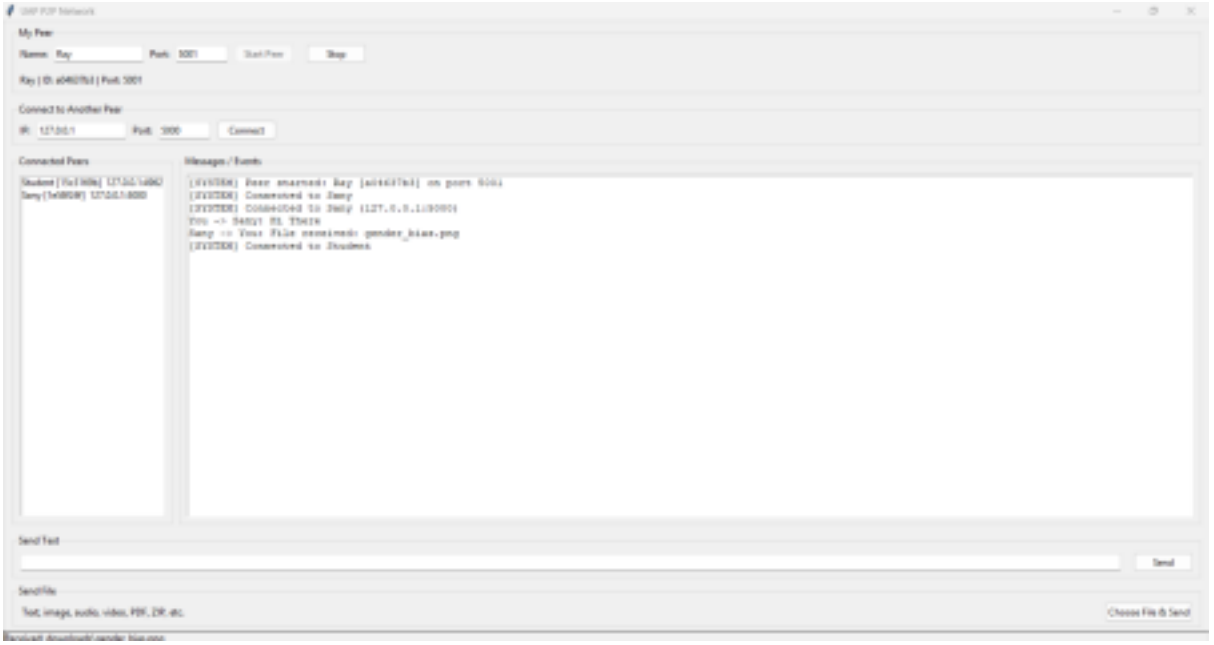


Figure 4: Peer 2

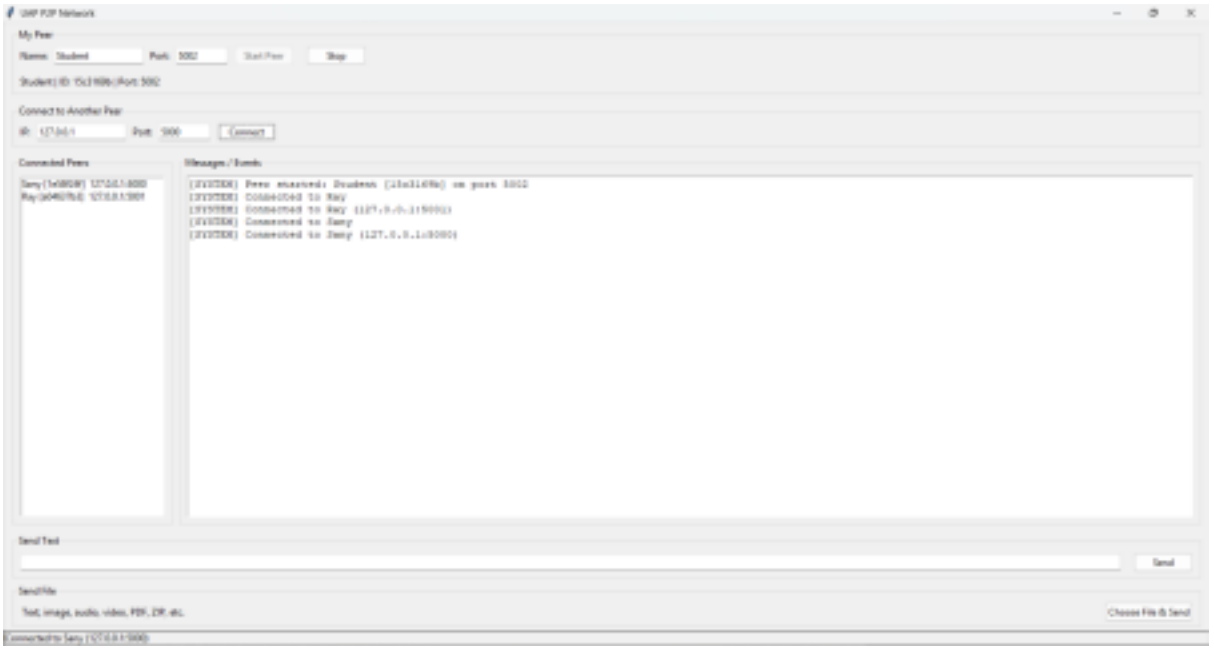


Figure 5: Peer 3

8. Why is TCP used instead of UDP?
9. Why are threads used?
10. What is the purpose of the HELLO message?
11. What is message framing?
12. Why can't we assume that one recv() call gives us one complete message? 13. Why is file metadata sent before the actual file?
14. Why are files transferred in chunks?
15. How does the receiver know when the file transfer has finished?
16. What happens if the other peer disconnects?
17. Where is the received file stored?
18. Is there a central server in this system?
19. What would happen if the central server in a traditional client-server system went offline? 20. What additional challenges would need to be solved to build a large-scale Internet P2P system?

## 21. Expected Learning Outcome

At the end of the assignment, students should be able to look at the following communication process and explain every stage:

Peer A

↓

IP Address + Port

↓
TCP Socket
↓
HELLO Handshake
↓
Application Protocol
↓
Text / File Data
↓
Peer B

## 22. Final Note

The purpose of this assignment is to build a small but functional P2P network and understand the under lying networking principles. Students are encouraged to implement the project incrementally:

TCP Server → TCP Client → HELLO Handshake → Text Communication → File Transfer → Multiple Peers

A working simple implementation is more valuable than an unnecessarily complicated implementation that you cannot explain.

> Do not make the project unnecessarily complicated. The primary objective is to understand P2P communication using TCP sockets.